

Spencer Collins

9/8/2025

Professor Alvino

Assignment 4: Big-O Analysis – Linked List vs . Array Seq. Search

Arrays in C++ require pre-allocated memory in order to use them. They store elements in contiguous locations in memory which results in faster access when the index for a specific element is known. Linked Lists store elements in non-contiguous memory locations, thus requiring each entry to have an associated pointer that references the next element in the list. Because linked lists store elements in non-contiguous locations in memory, they are used in applications where the size of the list is not known beforehand. For sequentially searching an array the best case is $O(1)$ and the average and worst case are $O(n)$ where “n” is the size of the array being searched. The best case for sequentially searching an array is $O(1)$ because if the index of an element is known, the computer can immediately go to that spot in memory due to all of the array elements being stored in one large block of memory. $O(n)$ is the worst case since the entire array would have to be traversed in order to find the element at the end and its index is not known. For sequential search on a linked list, the best case is $O(1)$ if the element is at the beginning of the list, and the average and worst case is $O(n)$, where “n” is the size of the list. For any element after the first one, time complexity is $O(n)$ due to none of the elements not being stored in contiguous memory locations, meaning none of the element indices can be known. If the element index is known, searching an array is more efficient than searching a linked list. Having 10 elements or 10,000 elements in an array and a linked list does not change the time complexity for searching both. The space complexity for searching an array is always $O(1)$, minus the variable used to iterate through the array. For a linked list, the space complexity is $O(1)$ for the iterative approach and $O(n)$ for the recursive approach. For the iterative approach, it is $O(1)$ since the list is being traversed in place and no other memory is needed while performing the search. For the recursive approach, the space complexity is $O(n)$ since as the list is being traversed, if the current element is the incorrect one, then the

function is recursively called on the next node. This recursive function call requires memory to be allocated to store the return value, and the function is called as many times as there are incorrect elements in a row, up to the length of linked list itself. This is why the preferred approach is the iterative one since the space complexity is always $O(1)$ for searching both an array and a linked list.